

Report on Capstone Project on Credit Analytics using Statistical Tools in Python



Submission Details	
Submitted by – Name	Sahana A
USN Number	P03KU24M015046
Class & Section	MBA 3 rd Sem ‘A’ Section
Course Instructor	Shareef A P
Date of submission	10-03-2026

Dept. of Management Studies (MBA)

SOUNDARYA INSTITUTE OF MANAGEMENT AND SCIENCE

(Recognized by Govt. of Karnataka and Affiliated to Bangalore University)

Accredited by NAAC with "B++" Grade



Importing necessary library

```
In [19]: #importing necessary Library required for the data and descriptive statistics
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [20]: #Importing the dataset
file_path = r"D:\PYTHON PRACTICALS\Datasets\credit_analytics_capstone_dataset.xlsx"
df = pd.read_excel(file_path)
df.head(10)
```

```
Out[20]:
```

	Customer_ID	Age	Gender	Employment_Type	Monthly_Income	Loan_Amount	Loar
0	1	59	Male	Salaried	79921	918004	
1	2	49	Male	Salaried	41162	209772	
2	3	35	Male	Self-Employed	48833	122146	
3	4	28	Male	Salaried	37206	1206130	
4	5	41	Female	Salaried	71747	996924	
5	6	59	Male	Salaried	110037	337321	
6	7	39	Female	Salaried	95827	1306354	
7	8	43	Female	Salaried	97195	1152915	
8	9	31	Female	Salaried	75387	885714	
9	10	31	Female	Self-Employed	45226	222142	

```
In [24]: df.shape
```

```
Out[24]: (8000, 14)
```

```
In [25]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8000 entries, 0 to 7999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Customer_ID           8000 non-null   int64
1   Age                   8000 non-null   int64
2   Gender                8000 non-null   object
3   Employment_Type       8000 non-null   object
4   Monthly_Income        8000 non-null   int64
5   Loan_Amount           8000 non-null   int64
6   Loan_Tenure_Months    8000 non-null   int64
7   Interest_Rate         8000 non-null   float64
8   Credit_Utilisation    8000 non-null   float64
9   Past_Loans            8000 non-null   int64
10  Delayed_Payments      8000 non-null   int64
11  EMI                   8000 non-null   float64
12  Credit_Default         8000 non-null   int64
13  Observation_Month     8000 non-null   datetime64[ns]
dtypes: datetime64[ns](1), float64(3), int64(8), object(2)
memory usage: 875.1+ KB

```

```

In [26]: #defining numeric columns
numeric_cols = ["Age", "Monthly_Income", "Loan_Amount", "Loan_Tenure_Months", "Interest_Rate"]
df[numeric_cols].describe()

```

```

Out[26]:

```

	Age	Monthly_Income	Loan_Amount	Loan_Tenure_Months	Interest_Rate
count	8000.000000	8000.000000	8.000000e+03	8000.000000	8000.000000
mean	40.174875	69839.739125	7.712722e+05	35.784000	13.486849
std	11.191323	28985.177144	4.166000e+05	16.853155	2.605936
min	21.000000	20036.000000	5.005900e+04	12.000000	9.000000
25%	31.000000	44054.750000	4.103750e+05	24.000000	11.250000
50%	40.000000	70363.500000	7.721690e+05	36.000000	13.460000
75%	50.000000	94997.000000	1.127079e+06	48.000000	15.760000
max	59.000000	119978.000000	1.499811e+06	60.000000	18.000000

Step 5: Categorical Variable Distribution

```

In [27]: #defining categorical columns
categorical_cols = ["Gender", "Employment_Type", "Credit_Default"]

for col in categorical_cols:
    print(f"\nPercentage Distribution for {col}")
    print(df[col].value_counts(normalize=True)*100)

```

Percentage Distribution for Gender

Gender

Male 50.0625

Female 49.9375

Name: proportion, dtype: float64

Percentage Distribution for Employment_Type

Employment_Type

Salaried 70.6875

Self-Employed 29.3125

Name: proportion, dtype: float64

Percentage Distribution for Credit_Default

Credit_Default

0 78.35

1 21.65

Name: proportion, dtype: float64

```
In [28]: df.groupby("Employment_Type")[numeric_cols].mean()
```

```
Out[28]:
```

	Age	Monthly_Income	Loan_Amount	Loan_Tenure_Months	Int
--	-----	----------------	-------------	--------------------	-----

Employment_Type

Salaried	40.174889	69496.641379	770808.508223	35.823873	
Self-Employed	40.174840	70667.124094	772390.405117	35.687846	

```
In [29]: df.groupby("Credit_Default")[numeric_cols].mean()
```

```
Out[29]:
```

	Age	Monthly_Income	Loan_Amount	Loan_Tenure_Months	Interes
--	-----	----------------	-------------	--------------------	---------

Credit_Default

0	40.072431	74893.083759	6.969777e+05	38.563497	13.4
1	40.545612	51552.000000	1.040139e+06	25.725173	13.5

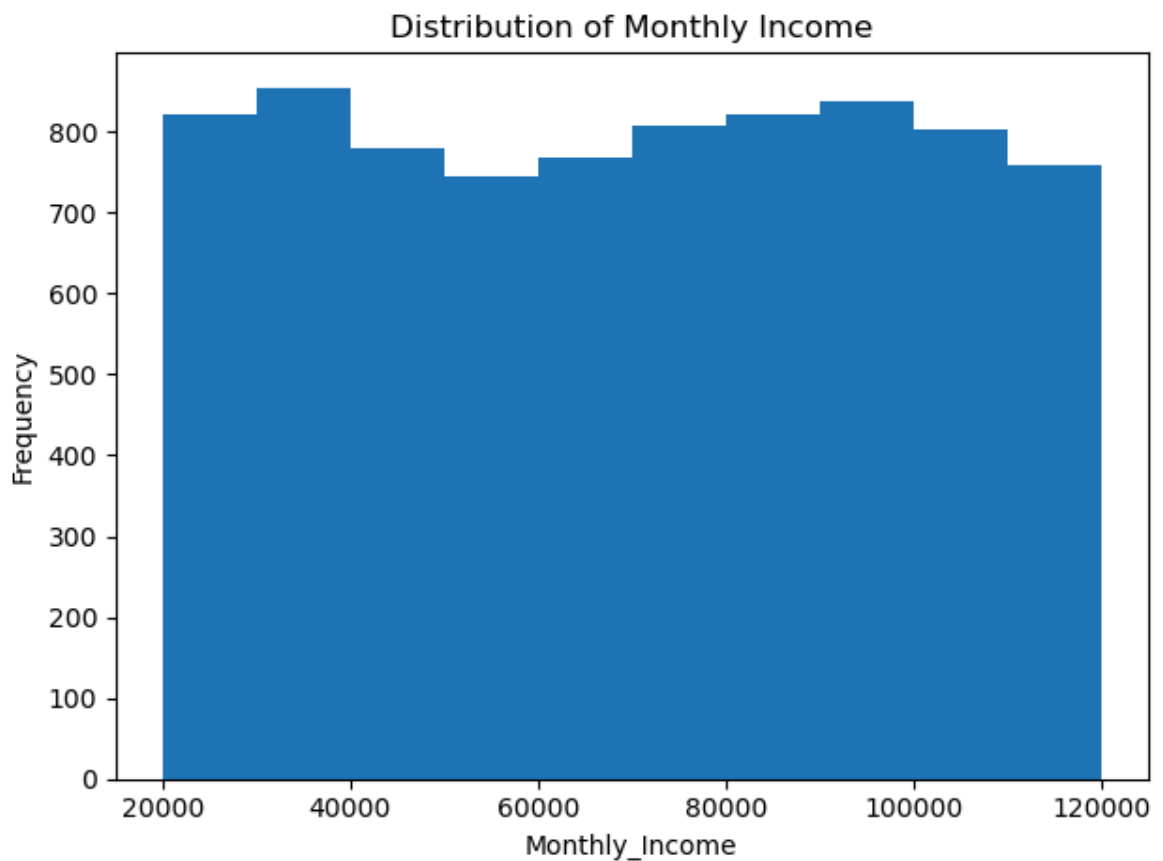
Step 7:Exploratory Visual Descriptives

Histogram-Monthly Income

```
In [30]: plt.figure()
plt.hist(df["Monthly_Income"],bins=10)
plt.xlabel("Monthly_Income")
plt.ylabel("Frequency")
plt.title("Distribution of Monthly Income")
plt.tight_layout()
plt.savefig("Histogram_Monthly_Income.png",dpi=300)
plt.show()
```

file:///C:/Users/SAHANA ANANDIAH/Downloads/Basicdata_descriptivestats.html

3/10



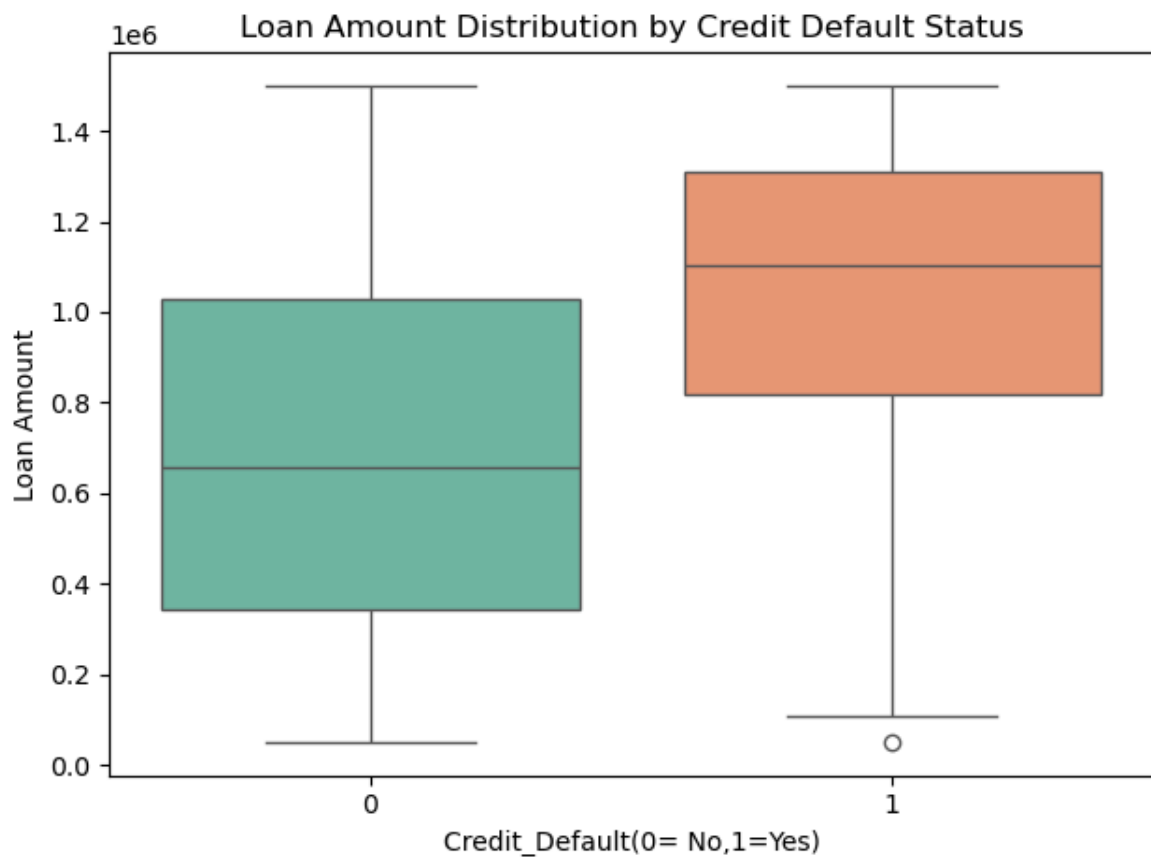
Boxplot-Loan Amount by Credit Default

```
In [31]: plt.figure()
sns.boxplot(x="Credit_Default",y="Loan_Amount",data=df,palette="Set2")
plt.xlabel("Credit_Default(0= No,1=Yes)")
plt.ylabel("Loan Amount")
plt.title("Loan Amount Distribution by Credit Default Status")
plt.tight_layout()
plt.savefig(r"D:\PYTHON PRACTICALS\Datasets\Boxplot_Loanamount_by_CreditDefault.")
plt.show()
```

C:\Users\SAHANA ANANDIAIAH\AppData\Local\Temp\ipykernel_24384\2394804715.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(x="Credit_Default",y="Loan_Amount",data=df,palette="Set2")
```



Task 2: Correlation & Covariance Analysis

Step 1: Select Variables for Relationship Analysis

```
In [32]: corr_cols = [
    "Monthly_Income" ,
    "Loan_Amount",
    "Loan_Tenure_Months",
    "Interest_Rate",
    "EMI",
    "Credit_Utilisation",
    "Delayed_Payments",
]
df_corr = df[corr_cols]
```

Step2: Correlation Matrix (Pearson)

```
In [33]: correlation_matrix = df_corr.corr()
correlation_matrix
```

Out[33]:

	Monthly_Income	Loan_Amount	Loan_Tenure_Months	Interest_Rate
Monthly_Income	1.000000	-0.011259	-0.001556	0.003137
Loan_Amount	-0.011259	1.000000	-0.034899	-0.008789
Loan_Tenure_Months	-0.001556	-0.034899	1.000000	0.013467
Interest_Rate	0.003137	-0.008789	0.013467	1.000000
EMI	-0.016957	0.703620	-0.582103	0.045011
Credit_Utilisation	-0.007499	0.004395	-0.003676	-0.000644
Delayed_Payments	-0.002805	-0.007809	-0.005640	-0.003270



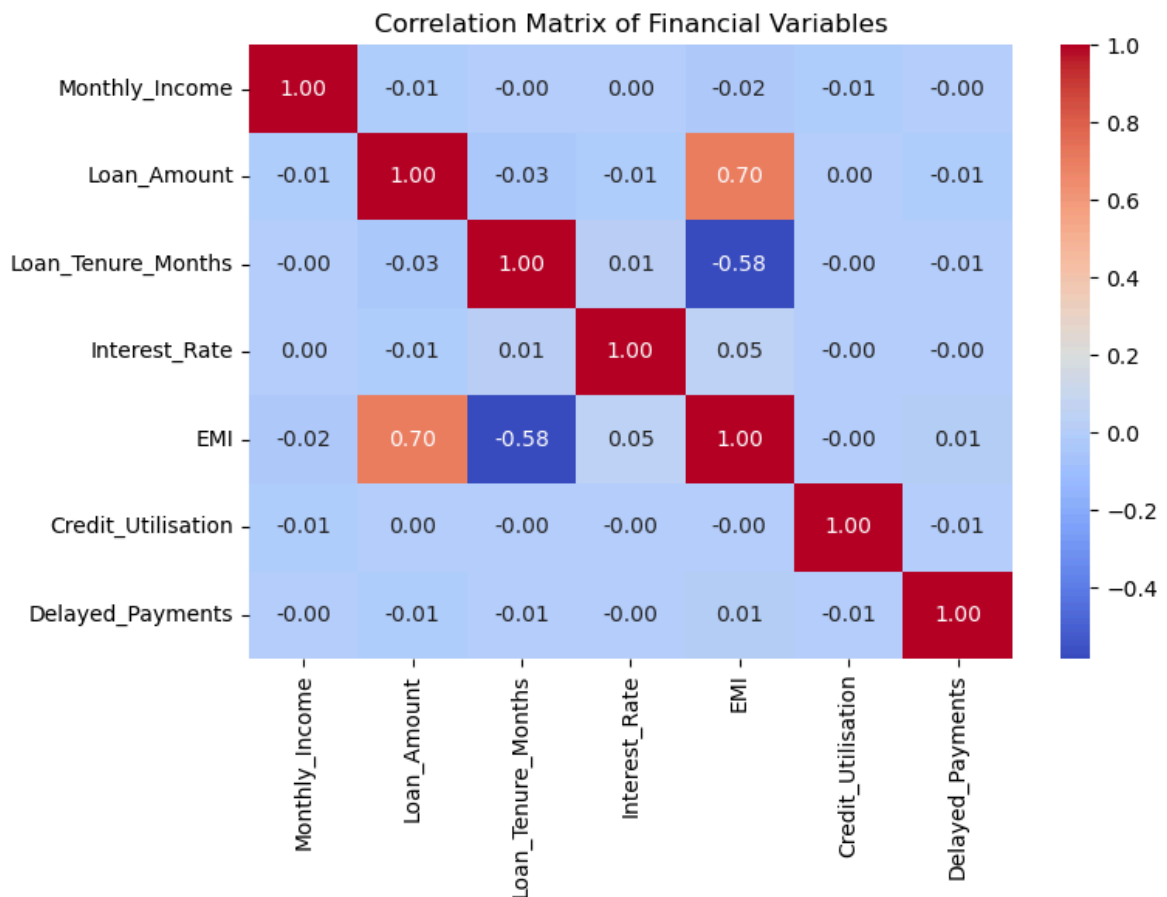
Step 4: Correlation Heatmap

```

In [34]: plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Matrix of Financial Variables")
plt.tight_layout()

# Save the Heatmap
plt.savefig((r"D:\PYTHON PRACTICALS\Datasets\Correlation_Heatmap.png"), dpi=300)
plt.show()

```



Task 3: Simple Linear Regression

Objective: Examine how Loan Amount influences EMI

Step 1: Select Variables

```
In [35]: x = df[["Loan_Amount"]] # Independent variable  
y = df["EMI"] # Dependent variable
```

Step 2: Import Required Library

```
In [36]: from sklearn.linear_model import LinearRegression
```

Step 3: Build and Fit the Model

```
In [39]: model = LinearRegression()  
model.fit(x, y)
```

```
Out[39]: ▼ LinearRegression ⓘ ?  
          ► Parameters
```

Step 4: Extract Regression Coefficients

```
In [40]: intercept = model.intercept_  
slope = model.coef_[0]  
  
intercept, slope
```

```
Out[40]: (np.float64(-809.653819306659), np.float64(0.05083525260215455))
```

Step 5: Model Fit (R-squared)

```
In [42]: r_squared = model.score(x, y)  
r_squared
```

```
Out[42]: 0.4950812592396051
```

Step 6: Predicted EMI

```
In [43]: df["Predicted_EMI"] = model.predict(x)  
df[["Loan_Amount", "EMI", "Predicted_EMI"]].head(10)
```


Out[43]:

	Loan_Amount	EMI	Predicted_EMI
0	918004	38428.667440	45857.311410
1	209772	10478.111400	9854.158790
2	122146	11324.969970	5399.668945
3	1206130	45454.349190	60504.269402
4	996924	51283.432100	49869.229546
5	337321	9993.134625	16338.144424
6	1306354	54587.453390	65599.181759
7	1152915	31484.187120	57799.071435
8	885714	37645.305320	44215.841104
9	222142	11893.852920	10482.990864

In [44]: `type(df)`

Out[44]: `pandas.core.frame.DataFrame`

In [45]: `df.info`

```

Out[45]: <bound method DataFrame.info of
Monthly_Income  Loan_Amount  \
0                1    59    Male    Salaried    79921    918004
1                2    49    Male    Salaried    41162    209772
2                3    35    Male    Self-Employed    48833    122146
3                4    28    Male    Salaried    37206    1206130
4                5    41    Female    Salaried    71747    996924
...            ...    ...    ...    ...    ...    ...
7995            7996    40    Female    Salaried    42134    1193004
7996            7997    49    Female    Salaried    119976    770120
7997            7998    48    Female    Salaried    28190    1324681
7998            7999    28    Female    Salaried    109364    1419218
7999            8000    44    Female    Self-Employed    24350    951149

Loan_Tenure_Months  Interest_Rate  Credit_Utilisation  Past_Loans  \
0                    36            16.90                0.42        1
1                    24             9.94                0.12        3
2                    12            11.26                0.40        2
3                    36            11.89                0.82        4
4                    24            11.73                0.51        1
...            ...            ...            ...    ...
7995                60            10.64                0.30        2
7996                36            10.78                0.85         0
7997                24            10.95                0.56        2
7998                12             9.14                0.44        4
7999                48            15.44                0.55        2

Delayed_Payments      EMI  Credit_Default  Observation_Month  \
0                0  38428.66744            0    2019-05-31
1                5  10478.11140            0    2021-10-31
2                5  11324.96997            0    2021-08-31
3                2  45454.34919            1    2022-11-30
4                1  51283.43210            0    2019-06-30
...            ...            ...            ...    ...
7995                0  30461.36880            0    2023-01-31
7996                3  28310.46689            1    2021-11-30
7997                0  67282.75579            1    2020-11-30
7998                1  129077.87710            0    2020-12-31
7999                3  32053.72130            1    2022-03-31

Predicted_EMI
0    45857.311410
1     9854.158790
2     5399.668945
3    60504.269402
4    49869.229546
...            ...
7995    59837.005876
7996    38339.590915
7997    66530.839433
7998    71336.651708
7999    47542.245858

[8000 rows x 15 columns]>

```

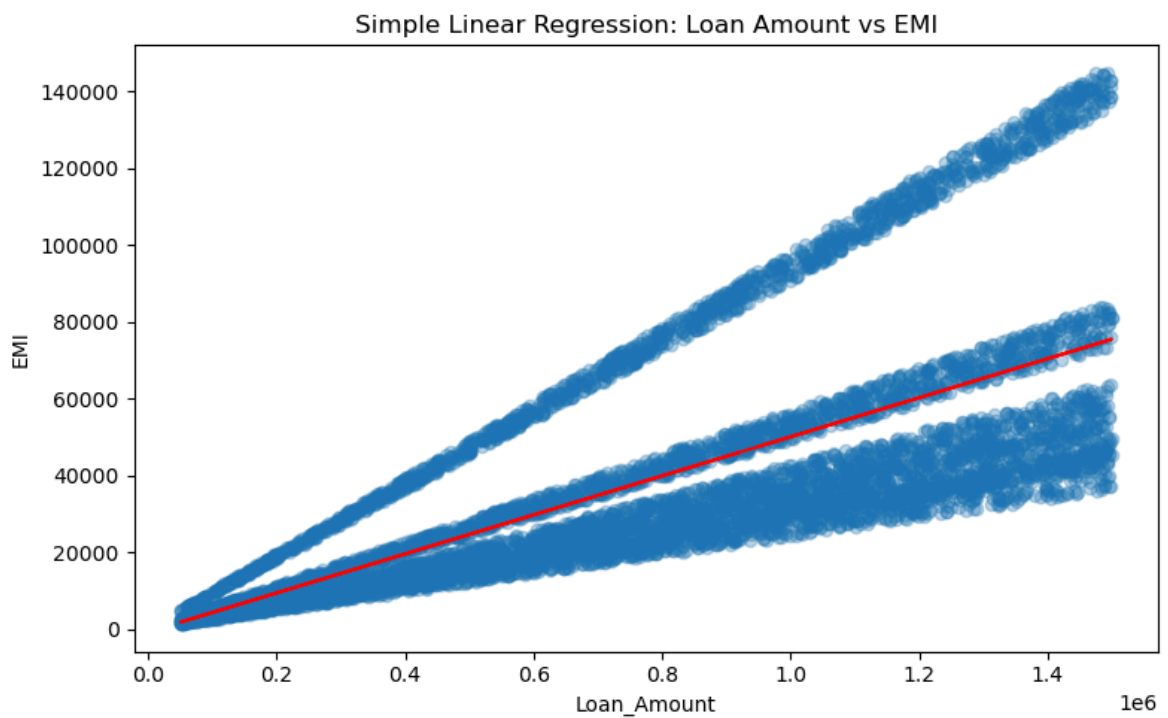
Step 7:Scatter Plot With Regression

```

In [48]: plt.figure(figsize=(8,5))
plt.scatter(df["Loan_Amount"],df["EMI"],alpha=0.3)

```

```
plt.plot(df["Loan_Amount"],df["Predicted_EMI"],color="red")  
plt.xlabel("Loan_Amount")  
plt.ylabel("EMI")  
plt.title("Simple Linear Regression: Loan Amount vs EMI")  
plt.tight_layout()  
plt.show()
```



In []: